

Course Work: Full-Stack Web Application Development

Coursework Assignment Brief

Overview

In this coursework you will design, build, and document a full-stack CRUD web application. You will choose a topic, either one of the four case studies provided, or a custom topic of your own and implement it end to end: a SQL database, a REST API server, and a client application, secured with an admin login

Learning Outcomes

- Design a relational database schema and implement it in a SQL database using the “Code-First Database Design” approach.
- Build a RESTful API using Express that performs full CRUD operations on related entities.
- Implement authentication and route protection to secure administrative functionality.
- Apply validation consistently on both the client and the server.
- Build a responsive, usable frontend using React or vanilla JavaScript, HTML and CSS.
- Organise a full-stack codebase following industry-standard project structure and version control practice.

Required Technology Stack

- **Backend:** Node.js with Express, structured as a REST API
- **Database:** A SQLite Database, accessed via a Sequelize ORM or parameterised raw SQL
- **Frontend:** A client application built using a React app created using Vite or vanilla HTML, CSS and JS
- **Authentication:** A JWT based admin authentication using username and hashed passwords

You are allowed to use additional libraries for validation, routing, HTTP clients etc., but core stack must be the same.

Choosing Your Topic

Select one topic from the table below, or propose a custom topic. A custom topic must involve at least two related entities of comparable complexity to the examples given, and must be approved by your tutor before you begin implementation. Find the topic list with requirements here: [📖 Topics with requirements](#)

Title	Core Entities	Distinctive Features
Inventory Management System	Products, Suppliers	Low-stock alert (red highlight below 5 units); supplier filter
Recipe Management System	Recipes, Ingredients	Ingredient selection with quantities per recipe; search by difficulty
Library Management System	Books, Authors, Genres	Low-stock alert on copies; author/genre reference dropdowns
Event Management System	Events, Attendees	Many-to-many attendee-to-event registration
Custom Topic (student proposed)	Student defined (min. 2 related entities)	Must be approved by the module tutor before development begins

Key Features Required for All Topics

Regardless of the topic chosen, every submission must implement the following core features.

CRUD of at Least Two Entities

- Full Create, Read, Update and Delete functionality for at least two related database entities
- Entities must be properly related in the database using foreign keys and relationships must be reflected in UI (e.g. a product shows its supplier's name, not just an ID)

User Entity & Authentication

- A dedicated user entity/table in the database separate from the business data entities
- A login form that authenticates the user against this table
- Unauthenticated users must not be able to view or modified protected data
- Passwords must be hashed
- Register functionality is optional, if multiple users are allowed make sure users can modify only their own data

Client-Side and Server-Side Validation

- Client-side validation should give immediate, user-friendly feedback (e.g. required fields, no negative numbers, valid email formats).
- Server-side validation is mandatory and must not be skipped: the server must never trust client input, since client-side checks can be bypassed.
- Clear, specific error messages must be returned and displayed when validation fails (not generic "error occurred" messages).

Proper Project Organization

- Separate client and server directories/projects with a clear folder structure (e.g. routes, controllers, models, config, public/src).
- Environment variables (e.g. database credentials, session secrets) stored in a .env file and excluded from version control via .gitignore.
- Use of Git with a meaningful commit history showing incremental progress, not a single final commit.

Responsive Frontend Design

- Layouts must adapt cleanly across mobile, tablet and desktop viewport widths (e.g. using CSS Flexbox/Grid or a responsive framework).
- Tables and forms should remain usable on small screens (e.g. stacking or horizontal scrolling rather than breaking).
- Navigation must remain accessible at all screen sizes (e.g. a collapsing menu on mobile).

Additional Required Features

The following are necessary to produce a complete, professional application and are also assessed:

- **Image upload:** Where the topic requires an image (product, book cover, recipe photo), users must upload an actual file from their device. A text URL field is not acceptable.
- **Search and filtering:** As specified per topic (e.g. search by name, filter by category/supplier/genre).
- **Status highlighting:** Where applicable to the topic, visually flag status for records e.g. Low Stock Alert for products.
- **RESTful API design:** Sensible, resource-based endpoints and HTTP verbs/status codes (e.g. GET /api/products, POST /api/products, 400 for validation errors, 401/403 for auth failures).
- **Error handling:** Server errors and failed requests should be handled gracefully, both in the API responses and in the frontend UI.

Topic-Specific Requirements

Topic specific requirements are provided for each case study in the full requirements document file.

Deliverables

1. Source Code: Full client and server side code submitted as a Git Repository Link
2. A report file: A report file to demonstrate what you implemented in the project. Your report should follow the guideline given here: [Report Guideline](#)

Assessment Criteria

The assessment of this Course-work will be done against the following criteria

	Criterion	Weight	What is assessed
Feature Implementation (40%)	Core CRUD Functionality	15%	Full Create, Read, Update, Delete for at least two related entities; correct database persistence; relationships between entities handled correctly.
	Authentication & Security	5%	Working login, hashed passwords, protected routes/middleware on both API and frontend, JWT handled correctly.
	Validation (Client + Server)	5%	Meaningful client-side validation for immediate feedback; matching server-side validation as the source of truth; clear error messages.
	Database Design	5%	Sensible schema, appropriate data types and constraints, foreign keys used correctly.
	Frontend Quality & Responsiveness	5%	Clean, usable UI; layout adapts correctly to mobile, tablet and desktop widths; sensible UX for lists, forms and alerts (e.g. low-stock highlighting).

	Code Organisation & Best Practice	5%	Clear folder structure, separation of concerns (routes/controllers/models), environment variables used for secrets, meaningful naming, version control history.
Source Control (10%)	Regular and meaningful commits pushed to the repository	10%	Git commits should be pushed on regular basis, with meaningful commit messages and content
Deployment (10%)	Working Deployment of the full-stack application	10%	The code should be deployed to a server. The frontend and backend should be accessible and the features of the application should work as expected
Demonstration (40%)	Live Demonstration	40%	You need to do a live 5-minute demonstration of your application during week 8. You need to demonstrate understanding of how you implemented the application.

Deadlines

Live Demonstration Deadline:

You need to give a live demonstration of your application during the **week 8 workshops**

Coursework Submission Deadline:

The coursework submission deadline will be on **August 16, 2026**

NOTE: These deadlines are final. Late submissions and extension requests cannot be accommodated.

Where to Submit?

You need to submit your coursework through **MST**. An assignment submission portal for **4SEWD-Coursework** is created for this assessment